

AI CMO Platform 전체 코드 감사 및 개선안

작성일: 2026-07-21

범위: 저장소 전체 코드, 워크플로 상태 전이, CLI/아티팩트 계약, 품질 게이트, 운영 경계, 스파게티 코드, 데몬/LSP 런타임, 외부 근거 조사

결론

현재 제품은 작은 로컬 YAML DAG + SQLite 실행기로서의 뼈대와 테스트 기반은 갖추고 있다. 그러나 실제 마케팅 산출물 파이프라인으로 신뢰하려면, 먼저 다음 네 가지를 고쳐야 한다.

- 의존 단계의 산출물을 다음 에이전트가 실제로 받도록 만든다.
- 실행 사양, 산출물 해시, 상태 전이, 이벤트를 하나의 원자적 계약으로 묶는다.
- phase-git을 안전하고 정직한 전달 흐름으로 재설계하거나 일시적으로 기능 표면에서 내린다.
- 문서상의 리뷰 기준을 실행 가능한 정책과 최종 산출물 게이트로 바꾼다.

중요한 판단: 지금 Temporal 같은 분산 워크플로 플랫폼으로 갈 필요는 없다. 단일 호스트 CLI의 현재 범위에서는 SQLite 트랜잭션, 사양 해시, 명시적 아티팩트 계보, 안전한 프로세스 관리만으로 우선순위 결함을 해결할 수 있다. 다중 호스트 워커, 장기 타이머/시그널, 높은 동시 쓰기, 강한 무중단 SLO가 생길 때 재검토한다.

감사 방법과 확인 범위

축	수행한 확인	결과
구조와 스파게티	import/DAG 순환, LOC/분기 집중, 중복 selector, 상태 계층, caller 추적	순환은 없었다. 넓은 리팩터보다 행동 결함 수리가 먼저다.
런타임	격리 SQLite/Git 재현, 실제 CLI 승인 흐름, 프로세스/리스너 스냅샷	핵심 상태/아티팩트 결함 8개를 재현했다.
품질	pytest, basedpyright, ruff, format, CLI help, dependency check	테스트는 통과했지만 타입/린트/포맷 게이트는 실패했다.
계약	YAML, README, SOP, 프롬프트, 실제 출력 경로, 테스트 공백	문서와 런타임 계약이 여러 지점에서 분리되어 있다.
외부 조사	SQLite, Python, Pydantic, Git, OWASP, NIST, OpenLineage, Temporal 자료	개선안을 최소 범위의 검증 가능한 통제로 좁혔다.

검증 결과:

- uv run pytest -q: 115 passed in 30.97s

- uv run basedpyright src tests: 5 errors
- uv run ruff check src tests examples: 11 findings
- uv run ruff format --check src tests examples: 7 files would be reformatted
- uv pip check: 22 installed packages compatible
- 실제 CLI smoke: approval-demo run -> waiting approval -> status -> approve -> resume -> success

우선순위 개선안

근거 분류: 재현은 격리된 SQLite/Git/adapter 또는 오류 주입으로 실제 경로를 실행한 결과이고, 실행 검증은 실제 CLI/품질 명령의 결과다. 정적 추적은 호출 경로를 소스에서 확인한 것으로, 과거 운영 사고가 있었다는 뜻은 아니다. 경계 미검증은 설계상 위험은 확인했지만 해당 외부 process tree를 아직 실제로 재현하지 않았다는 뜻이다.

P0 - 제품 정확성과 실행 안전성

ID	근거	문제와 영향	최소 개선	완료 검증
P0-1	재현: captured request	depends_on은 순서만 보장하고 downstream AgentRequest에는 upstream artifact가 없다. 전략, 리서치, 반영, 리포트가 서로의 결과를 모른 채 생성될 수 있다.	ArtifactRef를 도입한다. producer step, 상대경로, SHA-256, bounded content, contract version을 가진 입력 묶음을 명시적으로 다음 step에 전달한다.	첫 agent가 고유 토큰을 출력하면 둘째 agent의 캡처된 요청에 토큰, 경로, 해시가 모두 있어야 한다.
P0-2	재현: SQLite spec 변경	YAML 변경 후 resume이 false success를 만든다. 새 output은 생기지 않고 삭제한 실패 step은 ledger에 남아, 이전 실행이 현재 정의와 달라도 성공으로 표시된다.	run 생성 시 workflow YAML, 참조 prompt/role, 입력의 digest와 revision을 저장한다. resume은 동일 digest만 허용하고, migration은 명시적 reconcile 명령으로 분리한다.	output 추가/삭제, step 추가/삭제, prompt 변경 각각에서 resume이 안전하게 거부되거나 정책대로 재계산된다.
P0-3	재현: hash 삭제/변조	step success, output rows, hash, event가 분리돼 기록되고 hash가 없으면 변조 artifact를 정상으로 본다. 크래시 창에서 무결성 검사가 사라진다.	complete_step 트랜잭션 하나에서 status, outputs, hashes, event, artifact intent를 함께 commit한다. hash 누락은 unknown/fail-safe 상태로 처리한다.	hash 행 삭제 후 artifact 변조 시 resume이 성공하지 않고 repair 또는 명시 실패가 된다.
P0-4	재현: lease fault 주입	heartbeat renew 예외가 실행 경로에 전달되지 않아 lease 만료 뒤 다른 runner가 같은 step을 실행할 수 있다. 비용, 중복 외부 호출, 상충 산출물이 발생할 수 있다.	renew_lease는 bool 또는 typed result를 반환한다. heartbeat 오류는 cancellation token과 run failure로 전파하고, executor는 외부 write 전에 lease ownership을 재확인한다.	강제 renew 실패에서 원 runner가 외부 write 전에 중단되고, 두 runner가 성공하지 않는다.

ID	근거	문제와 영향	최소 개선	완료 검증
P0-5	재현: 격리 Git repo	phase-git은 artifacts와 .aicmo가 ignore된 상태에서 git add -A를 실행한다. 산출물 없이 성공을 알리거나 무관한 tracked 변경을 커밋할 수 있다.	기본은 off로 유지한다. 기능을 유지한다면 deliverable export를 별도 tracked 경로로 만들고, declared pathspec만 stage하며 empty commit을 오류/무변경으로 보고한다. run policy를 resume에도 보존한다.	무관한 변경은 index에 절대 올라가지 않는다. 생성 산출물이 없는 경우 성공 메시지가 나오지 않는다. resume도 동일 policy로 동작한다.

P1 - 품질, 보안, 운영 계약

ID	근거	문제	최소 개선	완료 검증
P1-1	정적 추적 + 재현: verdict parser	실제 reviewer는 SOP/gate-check를 읽지 않고 짧은 하드코드 계약만 사용하며 gate 뒤 final artifact도 존재한다. semantic verdict parser는 부분 문자열 매칭이라 NOT PASS와 PASSIVE를 PASS로 해석한다.	versioned reviewer policy와 JSON schema 결과, exact enum verdict를 도입한다. gate 입력 artifact 집합을 선언하고 final client-facing output 뒤 terminal gate를 둔다.	캡처된 reviewer request에 policy version, artifact refs, client 기준이 있다. NOT PASS, PASSIVE, 누락/모호 verdict는 reject/error이고, schema 오류는 한 번만 repair한 뒤 fail closed한다.
P1-2	정적 추적 + 재현: coercion/alias	입력/저장 JSON/onboarding/model alias가 느슨하게 coercion되거나 incidental error를 낸다.	strict Pydantic boundary model을 만든다. persisted JSON은 typed decode 실패를 quarantine/error로 분류한다. model alias는 allowlist/정식 ID 외 fail fast 한다.	list/object onboarding 값, array inputs_json, string outputs_json, 오타자 model alias가 명확한 validation error가 된다.
P1-3	정적 추적: DB event 경로	run input과 executor stderr가 SQLite 이벤트에 평문으로 남을 수 있다. 운영 노출 사고는 관찰하지 않았지만 pre-log 경계가 없다.	pre-log/pre-send deterministic redaction, sensitivity tag, 잘린 구조화 error를 도입한다. raw prompt/model output은 기본 telemetry에서 제외한다.	API key, bearer token, 이메일 형태 fixture가 DB/콘솔/telemetry 어디에도 원문으로 남지 않는다.
P1-4	정적 추적 + 경계 미검증	CommandAdapter timeout은 direct child 경계만 다루며 Windows descendant tree 정리는 설계되지 않았다. phase-git network 명령에도 timeout/비대화형 경계가 없다. child/grandchild 생존은 아직 실제 subprocess tree로 재현하지 않았다.	Windows process group/tree kill 전략과 bounded drain을 사용한다. Git에는 GIT_TERMINAL_PROMPT=0, timeout, 명시 retry 정책을 둔다.	fake child/grandchild가 timeout 뒤 남지 않는다. credential prompt와 hung remote가 정해진 시간 안에 typed failure가 된다.
P1-5	재현 + 정적 추적: error 주입	PNG renderer와 optional Anthropic client의 일부 시작/timeout 예외가 정리된 결과가 아니라 예외로 샌다. 실제 API 호출은 optional dependency 부재로 검증하지 않았다.	provider/renderer boundary를 typed unavailable/failed result로 정상화한다.	OSError, TimeoutExpired, client construction failure가 traceback 대신 operator-safe message와 structured event를 남긴다.
P1-6	실행 검증: local gates	basedpyright, ruff, format gate가 현재 red다. README의 테스트/step 수지도 오래됐다.	타입 오류와 lint/format을 고치고 CI에서 같은 명령을 필수화한다. README는 live count/계약에 맞춘다.	pytest, basedpyright, ruff check, ruff format check가 모두 0이다.

P2 - 사용성, 문서, 작은 구조 정리

ID	확인된 문제	권고
P2-1	artifacts per run과 outputs per client가 동시에 표준처럼 문서화돼 있다.	하나를 canonical delivery surface로 결정한다. 필요하다면 artifacts는 내부 run ledger, outputs는 export projection으로 명확히 구분한다.
P2-2	artifact-format은 실제 파일 형식 보장이 아니다.	제품 계약을 먼저 선택한다. prompt hint라면 명칭을 바꾸고, 보장이면 extension, serializer, parser 검증까지 구현한다.
P2-3	select_review_adapter는 select_adapter와 중복이다.	selector 하나로 통일해 약 12 LOC를 제거한다.
P2-4	cli.py/inbox/gate/store를 넓게 나누고 싶은 유혹이 있다.	지금은 보류한다. cli.py는 composition root이고 4개 step type은 공통 lease/output 경계를 공유한다. 두 번째 caller가 생기고 E2E 테스트가 있을 때만 추출한다.

스파게티 코드 판정

확인된 사실

- Python import cycle과 workflow DAG cycle은 발견되지 않았다.
- persistence 계층은 db -> run_state -> step_state -> ledger_state -> store의 단방향 구조다.
- cli.py와 step_executor.py는 크지만, 무작정 class/registry/event bus로 쪼개면 코드가 줄지 않고 호출 경로만 늘어난다.
- 가장 큰 문제는 파일 길이가 아니라 데이터 계보, 상태 원자성, 실행 정책이 서로 다른 경로에 분산된 것이다.

하지 말아야 할 리팩터

- WorkflowRunner를 composition으로 전면 전환하지 않는다. 생성 지점과 테스트 표면이 넓다.
- step type handler registry를 만들지 않는다. 현재 4개 type의 공통 lease/output guard를 깨뜨릴 가능성이 있다.
- phase callbacks를 범용 event bus로 일반화하지 않는다. 현재 production caller는 CLI 한 곳이다.
- Temporal, 큐, 마이크로서비스를 P0 해법으로 도입하지 않는다.

데몬, LSP, CodeGraph 진단

LSP

96초 동안 3회 프로세스 스냅샷에서 실제 language server 실행 파일은 0개였다. 별개의 LazyCodex LSP daemon은 처음에 하나가 있었으나 새 PID 없이 종료됐고, 실제 편집 후 뒤 생성된 새 daemon 하나는 20초 동안 같은 PID로 유지됐다.

설정상 LSP MCP는 비활성이다. 다만 편집 후 hook은 daemon client를 호출할 수 있다. 구현은 exclusive lock으로 이미 실행 중인 daemon 경쟁 인스턴스를 중단한다. 따라서 편집 후의 짧은 node 프로세스와 장기 daemon 중복을 구분해야 하며, 현재 관찰에는 중복/재생성 loop 증거가 없다.

CodeGraph

CodeGraph 한 묶음은 정상적으로 serve.js wrapper, npm shim, native `codegraph serve --mcp`, watchdog의 4개 프로세스로 구성된다.

초기 스냅샷은 5개였고, 최종 재확인에는 17개 묶음(총 68개 프로세스)이 남아 있었다. 16개 wrapper는 Desktop Codex 호스트(PID 7732), 1개는 별도 Codex CLI 호스트(PID 51156)의 직접 자식이며 20초 동안 수가 변하지 않았다.

OMO wrapper는 CODEGRAPH_NO_DAEMON=1을 설정하고 stdio MCP 연결마다 독립 server를 띄운다. 그러므로 CodeGraph 하나가 자기 자식을 재귀적으로 증식한 구조는 아니다. 다만 오래된 stdio 연결이 Desktop Codex 아래 누적됐으므로, 다중 client/session 보존인지 연결 종료 누락인지 운영 수명주기 문제로 추적해야 한다.

이 저장소의 --executor codex도 agent step마다 새 codex exec를 실행한다. runner는 순차 실행이므로 한 정상 run 안의 동시 묶음 다수는 기대값이 아니다. 여러 run, Codex client context, timeout 뒤 descendant 생존은 PID-세션 상관관계와 장기 count로 구분해야 한다. fan-out 중에는 CodeGraph를 필요할 때만 켜거나 호스트 연결 정리/공유 server를 별도 운영 과제로 둔다.

권장 실행 순서

0-3일

- P0-1 artifact handoff contract와 regression test를 만든다.
- P0-2/P0-3 spec digest 및 atomic completion transaction을 만든다.
- P0-4 heartbeat error propagation과 lease ownership test를 만든다.
- phase-git을 off로 고정하거나 P0-5의 scoped delivery contract를 완성한다.

1-2주

- 실행형 reviewer policy, terminal gate, strict result schema를 도입한다.
- strict boundary validation, secret redaction, provider/renderer error contract를 적용한다.
- Windows executor process-tree cleanup과 noninteractive Git timeout을 검증한다.
- basedpyright/ruff/format을 CI mandatory gate로 만든다.

이후

- canonical output/export contract와 문서를 통일한다.
- artifact format 기능을 실제 보장으로 만들지 여부를 제품 결정한다.
- selector 중복을 제거한다.
- multi-host/long-lived workflow 요구가 실제로 생길 때만 Temporal 또는 동급 플랫폼을 평가한다.

외부 근거와 적용 해석

- SQLite transaction documentation (https://www.sqlite.org/lang_transaction.html)과 atomic commit documentation (<https://www3.sqlite.org/atomiccommit.html>)은 상태, output hash, 이벤트를 한 transaction으로 묶는 판단을 지지한다.
- Python sqlite3 documentation (<https://docs.python.org/3/library/sqlite3.html>)과 Pydantic strict mode (https://pydantic.dev/docs/validation/latest/concepts/strict_mode/)는 경계 데이터의 명시적 transaction/validation을 지지한다.
- Python subprocess documentation (<https://docs.python.org/3/library/subprocess.html>)는 timeout 뒤 direct child 처리와 별도로 process-tree lifecycle을 설계해야 하는 근거다.
- Git add documentation (<https://git-scm.com/docs/git-add>)과 Git environment documentation (<https://git-scm.com/docs/git>)은 pathspec 기반 stage와 비대화형 Git 경계를 지지한다.
- OWASP Logging Cheat Sheet (https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html), OWASP LLM Verification Standard (<https://owasp.org/www-project-llm-verification-standard/LLMSVS-v1.0-en.html>), NIST GenAI RMF Profile (<https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>)은 redaction, typed output validation, human approval, provenance의 최소 통제를 지지한다.
- OpenLineage object model (<https://openlineage.io/docs/spec/object-model/>)은 run input/output과 design metadata를 분리해 artifact lineage를 모델링하는 참고 기준이다.
- Temporal AI engineering patterns (<https://go.temporal.io/platform-hub/ai-engineering/ai-patterns>)은 idempotency와 prompt/version awareness의 참고 사례다. 이 보고서는 Temporal 도입을 권고하지 않는다.

완료 기준

다음 기준을 모두 만족하면 P0/P1 개선을 완료로 볼 수 있다.

- downstream request에 명시된 upstream artifact refs가 전달되고, 관련 hash가 바뀌면 dependent step이 무효화된다.
- run spec/input digest가 현재 정의와 다르면 resume이 silent success를 만들지 않는다.
- terminal step success, output rows, hashes, event가 하나의 transaction으로 commit된다.

-
- heartbeat 실패, hash 누락, YAML spec drift, phase-git resume, noninteractive Git timeout이 모두 regression test로 고정된다.
 - reviewer 정책과 schema version이 artifact decision event에 남고, final deliverable은 gate 이후에만 성공 상태가 된다.
 - pytest, basedpyright, ruff check, ruff format check가 CI와 로컬에서 모두 통과한다.

감사 중 확인된 강점

115개 테스트가 통과했고 실제 CLI의 승인/재개 흐름도 작동했다. 경로 containment, atomic file write, manual reject terminality, stale-owner final CAS, KB queue idempotency는 현재 범위에서 확인됐으며, import/DAG cycle이 없고 broad abstraction 추가를 피해야 한다는 결론도 코드 근거가 있다.

이 강점은 유지하되, 현재의 조용한 성공과 분리된 계약을 먼저 닫는 것이 가장 낮은 위험의 다음 단계다.